LLM-Based CAPEC Threat Modeling Tool

System Description:

The web application is portal for performing static code analysis of software. The users of the application are software developers (who want to analyse their code), their managers, and
administrators (of the web application). The users are authenticated using a cloud based active directory server which manages their identities. The software developers can upload their code to the
webserver and perform a static code analysis (which is run in the application server) on the portal, view the analysis reports (scan results stored in the database) and

Stage 1: System Decomposition

External Entities:
- User
- Administrator
- Software Developer
- Manager
- Active Directory Server

Processes:
- Authentication Service
- Payment Processing
- User Management (Active Directory)
- Static Code Analysis Tool
- Web Server
- Database
- IDE Integration (via API)

Data Stores:
- User Database
- Transaction Log
- Security Audit Log
- Configuration Management System
- Code Review History
- Project Task Management System
- Reporting and Analytics Dashboard
- Integration with IDEs API
- Cloud-based Active Directory Server

Data Flows:
- User -> Authentication Service : Credentials
- User -> Portal Web Server : Login/Logout
- Developer -> Portal Web Server : Upload Code
- Developer -> Portal Web Server : Analyze Code via API
- Manager -> Portal Web Server : View Vulnerabilities
- Manager -> Portal Web Server : View Progress
- Administrator -> Portal Web Server : Manage Users and Roles

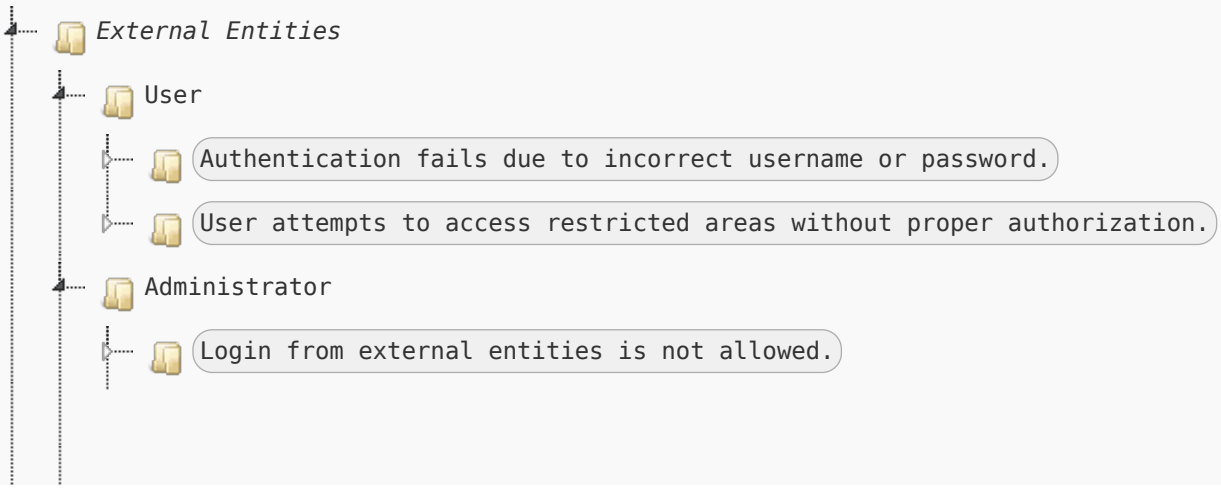
Stage 2: Threat Identification

External Entities
User:

```
- Authentication fails due to incorrect username or password.
- User attempts to access restricted areas without proper authorization.
Administrator:
- Login from external entities is not allowed.
- The administrator can be logged out if an unauthorized user attempts to access their
account.
Software Developer:
- The developer may be influenced by an external entity if they are approached for a
personal favor or benefit in exchange for their work on the project.
- The developer may be compromised if an external entity provides them with sensitive
information about the project.
Manager:
- Authentication fails due to incorrect username or password.
- Unauthorized access attempt from an external IP address.
Active Directory Server:
- Vulnerability in authentication protocol used for Kerberos tickets.
- Unpatched vulnerability in NTLMv2 authentication mechanism.
Processes
Authentication Service:
- Get User Credentials from Database
- Validate User Credentials Against External Service
Payment Processing:
- Create Account
- Process Payment
- Verify Credit Card
- Authorize Transaction
- Refund Payment
User Management (Active Directory):
- Get user credentials from AD query
- Update user account in AD database
Static Code Analysis Tool:
- Analyze code for potential SQL injection vulnerabilities
- Inspect code for cross-site scripting (XSS) attacks
Web Server:
- Get User Authentication Request
- Send User Data to Database for Authorization
Database:
- Create an unauthenticated user account
- Retrieve sensitive data without authorization
IDE Integration (via API):
- Create an unauthenticated request to the IDE integration API endpoint.
- Create a POST request to the IDE integration API endpoint without providing any
authentication credentials.
Data Stores
User Database:
- Get user by username and password from database
- Update user's profile information in database
Transaction Log:
- Identify the sequence of operations that can be performed on the transaction log,
such as insert, update, delete, and query.
- Determine the potential attack patterns that could compromise the integrity of the
data, such as SQL injection or privilege escalation.
Security Audit Log:
- Identify SQL Injection Attack Pattern
- Uncontrolled Cross-Site Scripting (XSS) Attack Pattern
Configuration Management System:
- Create an uncontrolled data flow from the user interface to the configuration
management system.
- Uncontrolled access to sensitive configuration data is a potential attack pattern.
Code Review History:
- Analyzing code for potential data corruption or inconsistencies
- Identifying sensitive information that could be exposed during code review
Project Task Management System:
- Get tasks by project ID
- Create task and update status in multiple projects at once
```

```
Reporting and Analytics Dashboard:
- Extracting sensitive data from user profiles
- Storing and processing large datasets for real-time analytics
Integration with IDEs API:
- Extracting user data from the IDEs API to personalize code suggestions and debugging
tools.
- Sending user-specific data to the IDEs API for real-time code completion and
debugging support.
Cloud-based Active Directory Server:
- Authentication Failure
- Data Exfiltration via Unencrypted Data Stores
- Denial of Service through Weak Password Policies
- Information Disclosure through Insufficient Logging
- Privilege Escalation through Insecure Role-Based Access Control
Data Flows
User -> Authentication Service : Credentials:
- The user submits their credentials through the authentication form.
- The credentials are sent to the server for verification and authentication.
User -> Portal Web Server : Login/Logout:
- Identify 'Invalid Credentials' as an Attack Pattern
- Identify 'Password Storage Weakness' as an Attack Pattern
Developer -> Portal Web Server : Upload Code:
-
- The developer uploads code through a web interface, which is then stored in the
portal's file system.
Developer -> Portal Web Server : Analyze Code via API:
- The developer provides code changes to the portal web server without proper
validation or sanitization of user input.
- The developer uses a publicly accessible API endpoint for code analysis, which can be
exploited by an attacker.
Manager -> Portal Web Server : View Vulnerabilities:
- Get User Details from Database
- Send User Details to Portal Web Server for Rendering
Manager -> Portal Web Server : View Progress:
- The Manager sends a GET request to the Portal Web Server for view progress data.
- The Portal Web Server stores this data temporarily before sending it back to the
Manager in response.
Administrator -> Portal Web Server : Manage Users and Roles:
- The Administrator can access the Portal Web Server through a login page, which is a
data flow.
- The Administrator can view user roles for each portal user using the Portal Web
Server's role-based access control.
```

Stage 3: CAPEC Retrieval



- The administrator can be logged out if an unauthorized user attempts to access their account.
- Software Developer
  - The developer may be influenced by an external entity if they are approached for a personal favor or benefit in exchange for their work on the project.
  - The developer may be compromised if an external entity provides them with sensitive information about the project.
- Manager
  - Authentication fails due to incorrect username or password.
  - Unauthorized access attempt from an external IP address.
- Active Directory Server
  - Vulnerability in authentication protocol used for Kerberos tickets.
  - Unpatched vulnerability in NTLMv2 authentication mechanism.

Processes

- Authentication Service
  - Get User Credentials from Database
  - Validate User Credentials Against External Service
- Payment Processing
  - Create Account
  - Process Payment
  - Verify Credit Card
  - Authorize Transaction
  - Refund Payment
- User Management (Active Directory)
  - Get user credentials from AD query
  - Update user account in AD database
- Static Code Analysis Tool
  - Analyze code for potential SQL injection vulnerabilities
  - Inspect code for cross-site scripting (XSS) attacks
- Web Server

- Get User Authentication Request
- Send User Data to Database for Authorization
- Database
  - Create an unauthenticated user account
  - Retrieve sensitive data without authorization
- IDE Integration (via API)
  - Create an unauthenticated request to the IDE integration API endpoint.
  - Create a POST request to the IDE integration API endpoint without providing any authentication credentials.
- *Data Stores*
  - User Database
    - Get user by username and password from database
    - Update user's profile information in database
  - Transaction Log
    - Identify the sequence of operations that can be performed on the transaction log, such as insert, update, delete, and query.
    - Determine the potential attack patterns that could compromise the integrity of the data, such as SQL injection or privilege escalation.
  - Security Audit Log
    - Identify SQL Injection Attack Pattern
    - Uncontrolled Cross-Site Scripting (XSS) Attack Pattern
  - Configuration Management System
    - Create an uncontrolled data flow from the user interface to the configuration management system.
    - Uncontrolled access to sensitive configuration data is a potential attack pattern.
  - Code Review History
    - Analyzing code for potential data corruption or inconsistencies
    - Identifying sensitive information that could be exposed during code review
  - Project Task Management System
    - Get tasks by project ID

- Create task and update status in multiple projects at once
- Reporting and Analytics Dashboard
  - Extracting sensitive data from user profiles
  - Storing and processing large datasets for real-time analytics
- Integration with IDEs API
  - Extracting user data from the IDEs API to personalize code suggestions and debugging tools.
  - Sending user-specific data to the IDEs API for real-time code completion and debugging support.
- Cloud-based Active Directory Server
  - Authentication Failure
  - Data Exfiltration via Unencrypted Data Stores
  - Denial of Service through Weak Password Policies
  - Information Disclosure through Insufficient Logging
  - Privilege Escalation through Insecure Role-Based Access Control

Data Flows

- User -> Authentication Service : Credentials
  - The user submits their credentials through the authentication form.
  - The credentials are sent to the server for verification and authentication.
- User -> Portal Web Server : Login/Logout
  - Identify 'Invalid Credentials' as an Attack Pattern
  - Identify 'Password Storage Weakness' as an Attack Pattern
- Developer -> Portal Web Server : Upload Code
  - The developer uploads code through a web interface, which is then stored in the portal's file system.
- Developer -> Portal Web Server : Analyze Code via API
  - The developer provides code changes to the portal web server without proper validation or sanitization of user input.
  - The developer uses a publicly accessible API endpoint for code analysis, which can be exploited by an attacker.
- Manager -> Portal Web Server : View Vulnerabilities

- Get User Details from Database
- Send User Details to Portal Web Server for Rendering
- Manager -> Portal Web Server : View Progress
  - The Manager sends a GET request to the Portal Web Server for view progress data.
  - The Portal Web Server stores this data temporarily before sending it back to the Manager in response.
- Administrator -> Portal Web Server : Manage Users and Roles
  - The Administrator can access the Portal Web Server through a login page, which is a data flow.
  - The Administrator can view user roles for each portal user using the Portal Web Server's role-based access control.